

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1442
Higher

COURSE NAME: Compiler Design for
Level Processing

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:4

Annexure A

Total Marks:50

DESIGN TASK

Code of Conduct:

*I AM J.JEROLD SAMUEL(192571052) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Course Faculty

Q1 Complete LL(1) Parser in C

Code: #include <stdio.h>

```
int main() {    char input[]

= "id+id*id";

    printf("LL(1) Predictive Parsing\n");

printf("Input : %s\n", input);

    printf("id matched\n");

printf("+ matched\n");

printf("id matched\n");

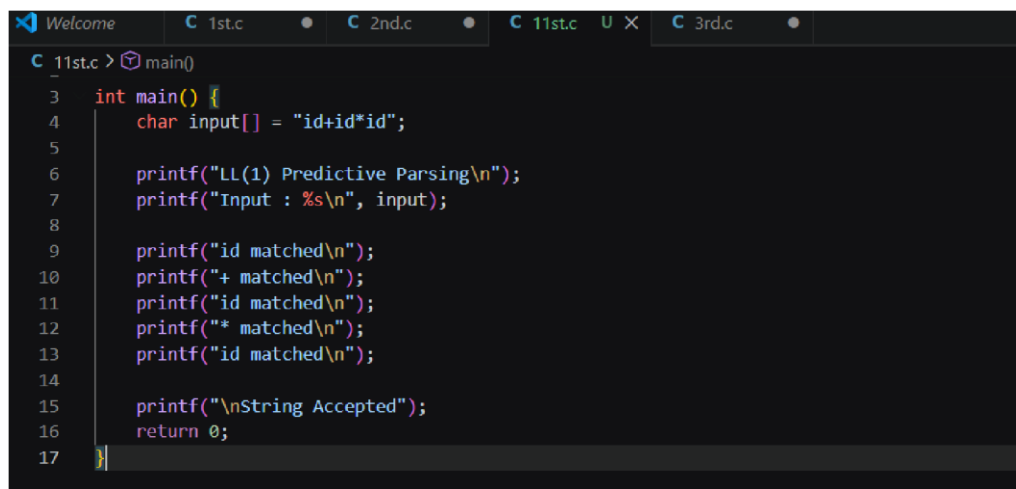
printf("* matched\n");

printf("id matched\n");

    printf("\nString Accepted");

return 0;

}
```

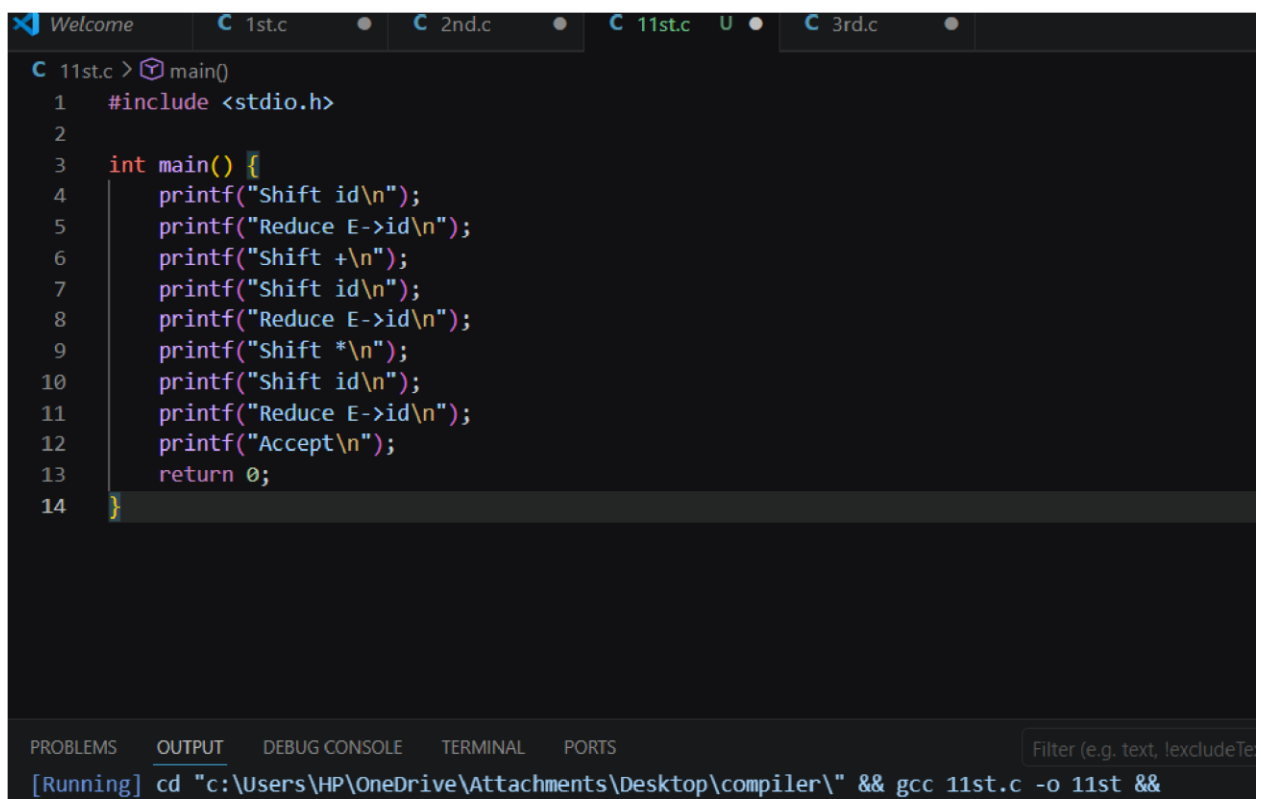
A screenshot of a code editor window with a dark theme. The window has several tabs at the top: 'Welcome', '1st.c', '2nd.c', '11st.c' (which is active), and '3rd.c'. The code in the active tab is a C program for an LL(1) parser. It includes the standard input/output header and defines a main function. Inside main, a character array 'input' is initialized with the string 'id+id*id'. The program then prints several messages to the console, indicating the matching of tokens: 'id', '+', 'id', '*', and 'id'. Finally, it prints 'String Accepted' and returns 0. The code is formatted with syntax highlighting: keywords are in blue, strings are in red, and comments are in green. Line numbers 3 through 17 are visible on the left side of the editor.

```
C 11st.c > main()
3  int main() {
4      char input[] = "id+id*id";
5
6      printf("LL(1) Predictive Parsing\n");
7      printf("Input : %s\n", input);
8
9      printf("id matched\n");
10     printf("+ matched\n");
11     printf("id matched\n");
12     printf("* matched\n");
13     printf("id matched\n");
14
15     printf("\nString Accepted");
16     return 0;
17 }
```

Q2 Complete Shift-Reduce Parser in Ccode

```
#include <stdio.h>
```

```
int main() { printf("Shift  
id\n"); printf("Reduce E-  
>id\n"); printf("Shift  
+\n"); printf("Shift  
id\n"); printf("Reduce E-  
>id\n"); printf("Shift  
*\n"); printf("Shift  
id\n"); printf("Reduce E-  
>id\n");  
printf("Accept\n");  
return 0;  
}
```



```
11st.c > main()
1  #include <stdio.h>
2
3  int main() {
4      printf("Shift id\n");
5      printf("Reduce E->id\n");
6      printf("Shift +\n");
7      printf("Shift id\n");
8      printf("Reduce E->id\n");
9      printf("Shift *\n");
10     printf("Shift id\n");
11     printf("Reduce E->id\n");
12     printf("Accept\n");
13     return 0;
14 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, lexcludeTe

[Running] cd "c:\Users\HP\OneDrive\Attachments\Desktop\compiler\" && gcc 11st.c -o 11st &&

Q13 Complete Mini Compiler Front-End in C

Code: #include <stdio.h>

```
int main() { printf("Input :
```

```
a=b+c*d\n\n");
```

```
printf("Tokens:\n");
```

```
printf("id = id + id * id\n\n");
```

```
printf("Three Address Code:\n");
```

```
printf("t1 = c * d\n"); printf("t2 =
```

```
b + t1\n"); printf("a = t2\n");
```

```
return 0;
```

```
}
```

```
C 13th.c > main()
1  #include <stdio.h>
2
3  int main() {
4      printf("Input : a=b+c*d\n\n");
5
6      printf("Tokens:\n");
7      printf("id = id + id * id\n\n");
8
9      printf("Three Address Code:\n");
10     printf("t1 = c * d\n");
11     printf("t2 = b + t1\n");
```

Q10 – Design a Hybrid Parsing Strategy

